

## 2. Práctica de POO

---

### Bloque 1 — Definir clases

#### Ejercicio 1

Definí una clase `Libro` que represente un libro. Al crear un libro, se deben poder indicar los siguientes atributos:

- `titulo` (string)
- `autor` (string)
- `anio` (int) — año de publicación
- `paginas` (int) — cantidad de páginas

Por ahora la clase **solo** debe tener su constructor. No hace falta agregar ningún método todavía.

Escribí también código que cree dos o tres libros distintos y que muestre por pantalla algunos de sus atributos usando la notación punto.

Ejemplo de uso:

```
>>> libro = Libro("El nombre del viento", "Patrick Rothfuss", 2007, 662)
>>> libro.titulo
'El nombre del viento'
>>> libro.autor
'Patrick Rothfuss'
>>> libro.paginas
662
```

#### Ejercicio 2

Definí una clase `Circulo` que represente un círculo a partir de un único atributo `radio`.

Agregá dos métodos que **devuelven** un número (no lo imprimen), usando estas fórmulas:

- `area()` : devuelve el área del círculo, que se calcula como  $\pi \cdot \text{radio}^2$ .
- `perimetro()` : devuelve el perímetro (la longitud de la circunferencia), que se calcula como  $2 \cdot \pi \cdot \text{radio}$ .

Podés usar `3.1416` como valor aproximado de  $\pi$ , o importar `pi` desde el módulo `math`.

Ejemplo de uso:

```
>>> c = Circulo(10)
>>> c.area()
314.16
>>> c.perimetro()
62.832
```

### Ejercicio 3

Definí una clase `Cancion` con los atributos `titulo`, `artista` y `duracion` (la duración en segundos, un entero).

Agregá el método especial `__str__` para que, al imprimir una canción, se muestre su título, su artista y su duración en formato `minutos:segundos`. Los segundos deben mostrarse siempre con dos dígitos.

Ejemplo de uso:

```
>>> cancion = Cancion("Bohemian Rhapsody", "Queen", 355)
>>> print(cancion)
Bohemian Rhapsody - Queen (5:55)
>>> otra = Cancion("Yesterday", "The Beatles", 125)
>>> print(otra)
Yesterday - The Beatles (2:05)
```

---

## Bloque 2 — Métodos: modificadores y puros

### Ejercicio 4

Definí una clase `Bateria` que represente la batería de un dispositivo. La batería tiene un atributo `nivel` que va de 0 a 100 y que, si no se indica al crearla, arranca en 100.

Agregá los siguientes métodos:

- `cargar(cantidad)` : aumenta el nivel de la batería en `cantidad`. El nivel **nunca** puede superar 100 (si se pasa, queda en 100).
- `usar(cantidad)` : disminuye el nivel de la batería en `cantidad`. El nivel **nunca** puede ser menor a 0. Si se intenta usar más de lo disponible, debe mostrar el mensaje `"Batería insuficiente"` y dejar el nivel sin cambios.

Estos métodos **modifican** el estado del objeto.

Ejemplo de uso:

```
>>> b = Bateria()
>>> b.nivel
100
>>> b.usar(30)
>>> b.nivel
70
>>> b.cargar(50)
>>> b.nivel
100
>>> b.usar(200)
Batería insuficiente
>>> b.nivel
100
```

## Ejercicio 5

Cuando trabajamos con objetos, podemos escribir funciones de dos maneras distintas:

- Una **función modificadora** cambia el objeto que recibe.
- Una **función pura** no toca el objeto original: crea y **devuelve un objeto nuevo** con el cambio aplicado.

Definí una clase `Producto` con los atributos `nombre` y `precio`.

Ahora, **fuera de la clase**, escribí dos funciones que apliquen un descuento (un porcentaje):

- `rebajar(producto, porcentaje)` : es una función **modificadora**. Baja el `precio` del producto que recibe según el porcentaje indicado, cambiando ese mismo producto. No devuelve nada.
- `con_descuento(producto, porcentaje)` : es una función **pura**. No modifica el producto que recibe; en su lugar, crea y **devuelve un nuevo `Producto`** con el precio ya rebajado.

Escribí código que muestre la diferencia entre ambas: después de llamar a `con_descuento`, el producto original conserva su precio; después de llamar a `rebajar`, el producto original quedó más barato.

Ejemplo de uso:

```
>>> remera = Producto("Remera", 1000)
>>> oferta = con_descuento(remera, 20)
>>> oferta.precio
800.0
>>> remera.precio      # el original no cambió
1000
>>> rebajar(remera, 20)
>>> remera.precio      # ahora sí cambió
800.0
```

## Bloque 3 — Igualdad y suma

### Ejercicio 6

Definí una clase `Color` que represente un color en formato RGB, con tres atributos: `rojo`, `verde` y `azul` (cada uno un entero de 0 a 255).

Agregá:

- El método `__str__`, para que al imprimir un color se muestre como `RGB(r, g, b)`.
- El método `__eq__`, de modo que dos colores se consideren iguales cuando sus tres componentes coinciden. Antes de comparar, verificá con `isinstance` que el otro objeto también sea un `Color`.

Ejemplo de uso:

```
>>> rojo = Color(255, 0, 0)
>>> print(rojo)
RGB(255, 0, 0)
>>> otro_rojo = Color(255, 0, 0)
>>> rojo == otro_rojo
True
>>> rojo is otro_rojo
False
>>> azul = Color(0, 0, 255)
>>> rojo == azul
False
```

### Ejercicio 7

Definí una clase `Fraccion` que represente una fracción, con atributos `numerador` y `denominador`.

El **denominador no puede ser cero**, ya que no está definida la división por cero. En el constructor, validá esta condición: si se intenta crear una fracción con denominador `0`, mostrará el mensaje `"El denominador no puede ser cero"` (o lanzá un error, si ya viste cómo hacerlo).

Para resolverlo vas a usar estas dos reglas:

- **Suma:**  $a/b + c/d = (a \cdot d + c \cdot b) / (b \cdot d)$ . Por ejemplo,  $1/2 + 1/3 = 5/6$ .
- **Igualdad de valor:** dos fracciones son iguales si representan la misma cantidad, aunque estén escritas distinto ( $1/2$  y  $2/4$  son iguales). Podés comprobarlo con la multiplicación cruzada:  $a/b == c/d$  cuando  $a \cdot d == c \cdot b$ .

Agregá:

- El método `__str__`, para mostrar la fracción como `numerador/denominador` (ej.:  $1/2$ ).
- El método `__eq__`, usando la multiplicación cruzada.
- El método `__add__`, para sumar dos fracciones con `+`, devolviendo una **nueva Fraccion**.
- Un método **puro** `a_decimal()` que devuelva el valor de la fracción como número decimal (ej.:  $1/2 \rightarrow 0.5$ ).

Ejemplo de uso:

```
>>> f = Fraccion(1, 2)
>>> print(f)
1/2
>>> f.a_decimal()
0.5
>>> print(Fraccion(1, 2) + Fraccion(1, 3))
5/6
>>> Fraccion(1, 2) == Fraccion(2, 4)
True
>>> mala = Fraccion(3, 0)
El denominador no puede ser cero
```

---

## Bloque 4 — Composición

### Ejercicio 8

Un objeto puede contener a otro objeto como atributo. A esto lo llamamos **composición**.

Definí dos clases:

- `Motor`, con los atributos `cilindrada` (float) y `potencia` (int, en caballos de fuerza).
- `Auto`, con los atributos `marca`, `modelo` y `motor`. El atributo `motor` **es un objeto de tipo `Motor`**.

Agregá un método `__str__` a `Auto` que muestre la marca, el modelo y los datos del motor.

Mostrá que se puede acceder de forma anidada a los atributos del motor desde el auto (por ejemplo, `auto.motor.potencia`).

Ejemplo de uso:

```
>>> motor = Motor(1.6, 110)
>>> auto = Auto("Renault", "Sandero", motor)
>>> auto.motor.potencia
110
>>> auto.motor.cilindrada
1.6
>>> print(auto)
Renault Sandero - Motor 1.6 (110 HP)
```

## Ejercicio 9

Reutilizando la clase `Cancion` del Ejercicio 3, definí una clase `Playlist` que represente una lista de reproducción.

Una `Playlist` tiene un `nombre` y una lista de canciones (que al crearse arranca vacía).

Agregá los siguientes métodos:

- `agregar(cancion)`: agrega una `Cancion` a la playlist.
- `duracion_total()`: devuelve la suma de las duraciones (en segundos) de todas las canciones.
- `__str__`: muestra el nombre de la playlist, cuántas canciones tiene y el listado de sus canciones.

Ejemplo de uso:

```
>>> p = Playlist("Para programar")
>>> p.agregar(Cancion("Bohemian Rhapsody", "Queen", 355))
>>> p.agregar(Cancion("Yesterday", "The Beatles", 125))
>>> p.duracion_total()
480
>>> print(p)
Playlist: Para programar (2 canciones)
- Bohemian Rhapsody - Queen (5:55)
- Yesterday - The Beatles (2:05)
```

## Bloque 5 — Atributos de clase

### Ejercicio 10

Recordá que los **atributos de clase** son compartidos por todas las instancias, mientras que los **atributos de instancia** son propios de cada objeto. Un buen uso de un atributo de clase es guardar un valor que vale **igual para todos** los objetos, como una regla o una constante.

Definí una clase `Estudiante` que tenga:

- Un **atributo de clase** `nota_para_aprobar`, con valor inicial `6`. Es el mismo para todos los estudiantes: representa la nota mínima con la que se aprueba.
- Atributos de instancia `nombre` y `notas` (una lista de notas, que arranca vacía).
- Un método `agregar_nota(nota)` que agregue una nota a la lista.
- Un método `promedio()` que devuelva el promedio de las notas del estudiante (o `0` si todavía no tiene notas).
- Un método `aprobo()` que devuelva `True` cuando el promedio del estudiante es mayor o igual a `nota_para_aprobar`.
- Un **método de clase** (`@classmethod`) llamado `cambiar_nota_para_aprobar(nueva_nota)` que cambie el atributo de clase. Al cambiarlo, la nueva exigencia debe valer para **todos** los estudiantes.

Ejemplo de uso:

```
>>> a = Estudiante("Ana")
>>> a.agregar_nota(6)
>>> a.agregar_nota(7)
>>> a.promedio()
6.5
>>> a.aprobo()
True
>>> Estudiante.cambiar_nota_para_aprobar(7)
>>> a.aprobo()          # ahora se exige 7
False
```

## Bloque 6 — Herencia

### Ejercicio 11

La **herencia** nos permite definir una nueva clase como una versión especializada de otra.

Definí una clase `Figura` que sirva como clase padre. Va a tener un método `area()` que, en la clase padre, simplemente devuelva `0` (las subclases lo van a redefinir).

Definí dos subclases de `Figura`:

- `Cuadrado`, con un atributo `lado`. Redefiní ( `override` ) el método `area()` para que devuelva el área del cuadrado.
- `Circulo`, con un atributo `radio` (podés reutilizar la clase del Ejercicio 2 adaptándola para que herede de `Figura`). Redefiní `area()` para que devuelva el área del círculo.

Agregá a cada subclase un método `__str__` que la describa.

Escribí código que cree una lista con figuras de distinto tipo y que recorra la lista imprimiendo el área de cada una.

Ejemplo de uso:

```
>>> figuras = [Cuadrado(5), Circulo(10)]
>>> for f in figuras:
...     print(f, "->", f.area())
Cuadrado de lado 5 -> 25
Círculo de radio 10 -> 314.16
```

## Ejercicio 12

Definí una clase `Usuario` que represente al usuario de un servicio de almacenamiento en la nube, con los atributos `nombre` y `espacio_base` (los GB incluidos). Agregá un método `espacio_total()` que devuelva el `espacio_base`.

Definí una subclase `UsuarioPremium` que herede de `Usuario`. Un usuario premium, además del espacio base, tiene un `espacio_extra`. Al escribir el constructor de `UsuarioPremium`:

- Usá `super().__init__(...)` para inicializar el `nombre` y el `espacio_base` reutilizando el constructor de la clase padre.
- Agregá el nuevo atributo `espacio_extra`.

Redefiní ( `override` ) el método `espacio_total()` en `UsuarioPremium` para que devuelva el espacio base **más** el espacio extra.

Ejemplo de uso:

```
>>> u = Usuario("Ana", 15)
>>> u.espacio_total()
15
>>> p = UsuarioPremium("Bruno", 15, 100)
>>> p.espacio_total()
115
>>> p.nombre
'Bruno'
```

## Bloque 7 — Integración y repaso

### Ejercicio 13 (Integrador)

En este ejercicio vas a combinar varios conceptos: herencia, composición, atributos de clase e igualdad.

Definí una clase `Persona` con los atributos `nombre` y `dni`. Agregá el método `__eq__`, de modo que dos personas se consideren iguales cuando tienen el mismo `dni`.

Definí dos subclases de `Persona`:

- `Estudiante`, que además tiene una `carrera`.
- `Docente`, que además tiene una `materia`.

Ambas subclases deben usar `super().__init__(...)` para inicializar `nombre` y `dni`.

Definí una clase `Escuela` que tenga un `nombre` y una lista de personas (por composición). La `Escuela` debe tener:

- Un **atributo de clase** `escuelas_creadas` que cuente cuántas escuelas se crearon.
- Un método `inscribir(persona)` que agregue una persona a la escuela. No se debe poder inscribir dos veces a la misma persona (usá la igualdad por `dni` que definiste).
- Un método `cantidad_de_personas()` que devuelva cuántas personas hay en la escuela.

Ejemplo de uso:

```
>>> escuela = Escuela("Escuela Central")
>>> ana = Estudiante("Ana", 40111222, "TUIA")
>>> bruno = Docente("Bruno", 28999888, "Programación 2")
>>> escuela.inscribir(ana)
>>> escuela.inscribir(bruno)
>>> escuela.inscribir(ana)      # mismo dni, no se agrega de nuevo
>>> escuela.cantidad_de_personas()
2
>>> Escuela.escuelas_creadas
1
```

## Ejercicio 14 (Depuración)

El siguiente código tiene **varios errores**. Encontralos, explicá por qué están mal y proponé una versión corregida.

```
class Contador:
    total = 0

    def __init__():
        self.valor = 0

    def incrementar(self):
        valor = valor + 1
        Contador.total = Contador.total + 1

    def mostrar(self):
        print("Valor:", self.valor)

c = Contador()
c.incrementar()
c.mostrar()
```

Pistas: prestá atención al primer parámetro del constructor, a cómo se accede a los atributos de instancia dentro de un método, y a la diferencia entre una variable local y un atributo.

## Ejercicio 15 (Repaso guiado)

Volvé sobre algunas de las clases que ya definiste y extendelas un poco. Elegí y resolvé al menos dos de las siguientes:

1. Agregá a la clase `Libro` (Ejercicio 1) un atributo de instancia `paginas_leidas` (que arranque en 0) y un método modificador `leer(cantidad)` que sume páginas leídas sin superar el total de `paginas`.
2. Agregá a la clase `Playlist` (Ejercicio 9) un método `mas_larga()` que devuelva la canción de mayor duración de la playlist.
3. Agregá a la clase `Color` (Ejercicio 6) un método `es_gris()` que devuelva `True` cuando las tres componentes (rojo, verde y azul) son iguales entre sí.
4. Agregá a la clase `Bateria` (Ejercicio 4) un método `esta_cargada()` que devuelva `True` cuando el nivel es mayor a 20, y `False` en caso contrario.

En cada caso, escribí también unas líneas de código que prueben lo que agregaste.